

Fruit Slice - developer guide

Read `docs/GUIDE.md` (the engine and stack guide) first for the technology and workflow. This file is about this game: how it plays, how the code is laid out, how each system works, and how to change it. `GAME.md` is the short summary; this is the long version. Art tooling is in `art/CHATGPT-PROMPT.md`.

- Branch `fruitslice`, folder `FruitSlice`, app ID `com.maxmartin.fruitslice`
- Saved data (localStorage): `fruitslice.profile` (coins, skins, best scores, settings), `fruitslice.catalog` (last good online skin catalog)
- Dev URL flags: `?seed=N` reproducible run · `?time=S` Arcade start time · `?catalog=URL` online skins (put it first in the query) · `?gallery` shows all 19 fruit · `?ads=no-fill` (engine) no ads

1. The game in one page

A fruit sits on the field. The player drags **one straight line fully across it** (start outside the fruit, end outside the fruit, and the line crosses the fruit in between). The fruit splits in two; how close the halves' **areas** are to 50/50 decides the score. A drag that doesn't go fully across (starts or ends on the fruit, misses it, or is a tap) is a **cancel**: nothing happens, try again.

- **Tolerance** = how far from 50/50 a cut may be, in percentage points. 12 means 38/62 is fine.
- **Deviation** of a cut = $|\text{left share} - 50|$ in percentage points. Rating: Perfect ≤ 0.5 , Great ≤ 1.5 , otherwise Good. Accuracy = $100 - 2 \cdot \text{deviation}$. Points = accuracy $\times$ combo multiplier.
- **Combo**: a Perfect adds 1, a Great keeps it, anything else resets it. Multiplier = $1 + 0.1 \times \text{combo}$ (max 10).

Mode	Idea
Classic	One still fruit at a time, no clock. Tolerance starts at 12 and shrinks smoothly to 1.5 over 30 fruit; fruit also get smaller. Going over the tolerance ends the run.
Arcade	60 s clock (max 99), 3 strikes. A good cut adds 4 s (+1 for Perfect). A miss or a bomb costs a strike and 2 s. Fruit start still, then drift (from round 6), spin (12), bombs appear (16), two fruit at once (22). Slower tolerance curve (16 $\rightarrow$ 4 over 60 fruit).
Survival	You have a margin (25 to start, max 35). Every cut takes <code>deviation $\times$ lossMultiplier</code> off it (a perfect cut is free). A cut the margin can't pay for ends the run. Floating " +x " bubbles sit over the fruit; drag through one on a cut that holds and x is added. After 8 fruit the loss multiplier grows (+0.08 per fruit, cap $\times 4$) so nobody lasts forever.

After a failed run - "try again": pay coins (20, then $\times 2$ each time, cap 1000) or watch a rewarded ad (works for any try). The same fruit comes back. In Arcade you get time back (20 s) and keep 1 strike. In Survival the margin returns to what it was before the fatal cut. **End of run**: coins = $2/\text{fruit} + 3/\text{perfect} + \text{score}/100$. A rewarded ad doubles them (once). Returning to the menu shows an interstitial, but never after a rewarded ad that run, and at most every 45 s. **Shop**: knife skins (cosmetic only) bought with coins; the bundled "Steel" is free.

2. File map

```
src/game/
  index.ts      THE FLOW. phases: menu / playing / paused / failing / tryAgain / result / shop
  /
                settings / ad. Owns the profile, ads, sound, haptics, drag capture, events ->
UI.
  info.ts       GAME_ID and the localStorage keys
  catalog.ts    loads the online skin catalog (never blocks, never throws)
  sfx.ts        named sound effects built from engine.audio.tone()/noise()
  sim/
    config.ts   EVERY tuning number: tolerance curves, arcade, survival, rating, scoring,
economy
  geometry.ts   polygon maths: area, clip by a line, split areas, line crossings, bisecting
line
  cut.ts        evaluateCut(): is this drag a cancel or a cut, and with what deviation
  fruit.ts      the 19 fruit outlines (radial shapes), makeFruitShape, pickKind
  rules.ts      toleranceFor, fruitRadiusFor, rating, accuracy, combo, points, coins,
                restartCost, shouldShowInterstitial, lossMultiplierFor, survivalAllowed
  run.ts        THE RUN: createRun, stepRun, restartRun, endRun, events, spawning, modes
  profile.ts    saved data: coins, owned skins, equipped, best scores, settings (+
parse/serialize)
  skins.ts      skin data type, 10 bundled skins, validation of online ones, blade outlines
  index.ts      public exports of the sim
  view/
    scene.ts    field, fruit, bombs, "+x" bubbles, cut halves, juice, slash, the knife; showCut
etc.
  fruitSprites.ts loads the painted fruit sheet src/game/art/fruits.png and makes each fruit a
sprite
  palette.ts    colours for the UI and every fruit
  ui/
    ui.ts       menu, HUD, pause, try-again, result, shop, settings, flash popups
    skinPreview.ts draws a knife skin on a small canvas for the shop
    styles.css  look; dom.ts small element helpers
tests/
  geometry, fruit-rules, run, survival, meta (profile/skins/catalog) + engine
tests
e2e/
  fruitslice.spec.ts (every flow + screenshots), helpers.ts, engine.spec.ts
scripts/fruit-art.mjs  sprite sheet tools (guide + converter)
art/
  fruit-guide.png + CHATGPT-PROMPT.md
```

3. The geometry (how a cut is judged)

All in `sim/geometry.ts` and `sim/cut.ts`, on plain `{x, y}` points.

1. A fruit is a **polygon** (64 points): `place(outline, x, y, rotation, radius)` turns its unit-space outline into world coordinates. The drawing is made from the same outline, so what you see is what is measured.
2. **Is the drag fully across?** `lineCrossings(poly, a, b)` finds where the line segment $a \rightarrow b$ crosses the polygon's edges. The cut is valid only if **both ends are outside the fruit** and **every crossing lies inside the drag** (so the line really goes through it). Otherwise it's a cancel with a reason: `short` (a tap), `miss`, or `partial` (an end was on the fruit).
3. **How even is it?** `splitAreas(poly, a, b)` clips the polygon to each side of the infinite line through a and b (Sutherland-Hodgman half-plane clipping - works for concave shapes like the banana and starfruit) and returns the two areas. $\text{fraction} = \text{left} / (\text{left} + \text{right})$; $\text{deviation} = |\text{fraction} - 0.5| \times 100$.
4. `bisectingLine(poly, angle)` (used by tests and the e2e helper) does the opposite: finds the line at a given angle that splits the area exactly in half, by bisection search.
5. Two fruit at once (Arcade twins): *both* must be cut by the one line; the worst deviation counts.
6. Bombs: `segmentHitsCircle(a, b, bomb, r)` - if the drag touches a bomb, the cut is voided and costs a strike.

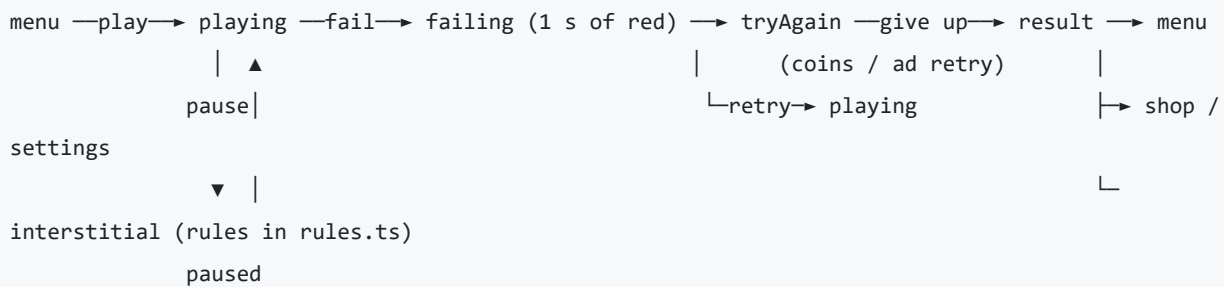
4. The run (`sim/run.ts`)

`Run` is a plain object: `mode`, `seed`, `rng`, `tick`, `phase` (`playing` | `failed` | `over`), `fruits`, `bombs`, `bubbles`, `round` (fruit won so far), `score`, `combo`, `tolerance` (allowed deviation for the fruit on screen), `arcade` `timeLeft/strikes`, `survival` `margin`, `restarts`, and `events`.

- `createRun(mode, seed)` → spawns the first round. `spawnRound(run)` places the next fruit (and bombs / bubble) using `run.round` for difficulty and the seeded `run.rng` for variety.
- `stepRun(run, { cut })` is called every fixed step with the drag that just ended (or null). It moves things (Arcade), counts down the clock, spawns after the cooldown, and calls `handleCut`.
- `handleCut` → bombs, then `evaluateCut` per fruit, then the verdict by mode (see the code comments).
- **Events** are the only way the sim talks to the outside: `spawn`, `cancel`, `cut`, `bomb`, `strike`, `margin`, `failed`, `restarted`, `over`. `index.ts` drains them every step (`drainEvents`) and turns them into popups, sounds, haptics, particles. The sim never calls the UI.
- `run.snapshot` stores the fruit/bombs/bubbles as spawned so **try again** brings back exactly the same ones. `restartRun` restores it (+ `margin` / `time` / `strikes` depending on the mode).
- Randomness: only `run.rng` (a seeded generator); `Math.random` is forbidden in `sim/` and a test enforces it.

5. The flow (`index.ts`)

The phase machine decides what is shown and what input means:



- Drag capture: `input.onGesture` collects start→end; on `end` the line (a, b) in **world** coordinates is passed to the next `stepRun` as `cut`. While dragging, the scene draws the glowing line + knife.
- `handleEvent(event)` is a big `switch` on the sim events (read it top to bottom to see all effects).
- Money: `profile` (pure functions `earn/spend/buySkin/equipSkin/recordBest`) is saved to `localStorage` after every change (`persist()`).
- Ads: rewarded for retries and doubling coins; interstitial after `result→menu` per `shouldShowInterstitial`. Everything goes through the engine's ad service, which is fake in a browser.

6. Drawing

- `scene.update({run, alpha, now})` each frame: keeps a `Map` of sprites by id for fruit, bombs and bubbles (create on appear, interpolate with `alpha`, destroy on disappear).
- A cut creates **two halves**: the *same* fruit picture twice, each hidden by a half-plane mask on one side of the cut line, then pushed apart, rotated and faded (`HALF_SECONDS`). Plus juice particles in the fruit's colours, a slash line and, for a miss, a red tint.
- The **knife**: a block drawn beside the drag line with the middle of its edge on the line, tilted and swaying; it pops in on touch and swishes on after release (`ghosts`).
- Fruit art: the painted sheet `src/game/art/fruits.png` (see section 9). `fruitSprites.ts` makes a sprite per fruit, scaled by the fruit's radius. If the file were missing the fruit would be drawn as plain flat silhouettes and the console would warn, so play never breaks.
- Pixi v8 `Graphics` API reminder (used for the bubbles, bombs, knife and effects):
`g.poly(points).fill(color).stroke({width, color}), g.circle(x, y, r),`
`g.moveTo().lineTo().stroke(). g.clear()` empties it.
- Pixi v8 `Graphics` API reminder: `g.poly(points).fill(color).stroke({width, color}), g.circle(x, y, r), g.ellipse(...), g.moveTo().lineTo().stroke()`. Calling `g.clear()` empties it.

7. Skins (the online cosmetic part)

A skin is **pure data**, never code or images:

```

{ "id": "ember", "name": "Ember", "rarity": "rare", "price": 300,
  "blade": { "color": "#ff7043", "edge": "#ffe0b2", "shape": "straight" },
  "handle": { "color": "#3e2723", "accent": "#ff3d00" },
  "trail": { "color": "#ff7043", "glow": "#ffab91", "width": 3 },
  "sparks": { "color": "#ffcc80" } }

```

`shape` is one of `straight | cleaver | curved | serrated`; `rarity` is `common | rare | epic | legendary`. 10 skins are bundled in `sim/skins.ts`. An online catalog is just a JSON file `{ "skins": [ ... ] }` hosted anywhere (even a raw file on GitHub / GitHub Pages). To use one: pass `?catalog=https://.../skins.json` in dev, or for a release set `VITE_SKINS_CATALOG_URL=https://.../skins.json` in `.env.production`. The game:

1. loads the cache first (instant), then downloads in the background with a 4 s timeout;
2. validates every skin (`parseSkin` in `sim/skins.ts` - read it for the exact rules) and silently drops bad ones; a broken file changes nothing;
3. merges online over bundled by `id` (an online skin with a bundled skin's id replaces it).

No server code is needed: a static JSON file is the "server". Add a skin by adding an object to it.

8. Tests

`npm run check` = typecheck + ~226 unit tests (fast):

File	Covers
<code>geometry.test.ts</code>	areas, clipping (convex and concave), crossings, bisecting line
<code>fruit-rules.test.ts</code>	every fruit outline is valid (polygon, normalised, not self-intersecting), tolerance/size curves, rating, scoring, combo, coins, restart cost, interstitial pacing
<code>run.test.ts</code>	cancels, cuts, strikes, bombs, Arcade chaos, restart, determinism
<code>survival.test.ts</code>	margin costs, bubbles, cap, loss multiplier, retry restores
<code>meta.test.ts</code>	profile saving/loading, shop rules, skin validation, catalog merging
<code>architecture.test.ts</code>	the layer rules

`npm run e2e` plays every flow in a browser (menu, each mode, cancel/perfect/miss, try again with ad and coins, doubling, interstitial pacing, shop + reload, settings, pause, online skins incl. broken ones, a wide desktop window, the fruit gallery) and saves screenshots to `e2e/screenshots/`. `e2e/helpers.ts` has `cutFruit(page, deviation, angle)`: it reads the real fruit polygon from `window.__game` and drags a line that splits it by exactly that deviation - copy this idea for any precise-input test.

9. The fruit art (painted sprites)

See `art/CHATGPT-PROMPT.md` for the step-by-step. In short: attach `art/fruit-guide.png` to ChatGPT with the prompt, save its picture, run `node scripts/fruit-art.mjs prepare <picture.png>`; the game then uses `src/game/art/fruits.png`. The original generated pictures are kept in `art/` (`chatgpt-sheet-2.webp`).

10. How to change things

I want to...	Change
Make Classic easier/harder	<code>config.tolerance.classic</code> (start/floor/fruitsToFloor), <code>config.fruit</code> (size curve)

I want to...	Change
Tune Arcade	<code>config.arcade</code> (clock, strikes, when each chaos starts, speeds)
Tune Survival	<code>config.survival</code> (start/max margin, bubble chance/size/value, when and how fast the loss multiplier grows)
Rate/score differently	<code>config.rating</code> , <code>config.scoring</code> , <code>rules.ts</code>
Pay more/less coins, change retry price	<code>config.economy</code>
Add a fruit	kind + <code>RadialSpec</code> in <code>sim/fruit.ts</code> (its invisible cut outline), juice colours in <code>view/palette.ts</code> , a 20th painted cell (regenerate the guide with <code>node scripts/fruit-art.mjs guide , paint, prepare</code>); <code>?gallery</code> to check; the "every kind ... is a valid polygon" unit test must pass
Add a skin	add to <code>BUNDLED_SKINS</code> or to the online JSON
Add a game mode	add to <code>Mode / MODES</code> in <code>config.ts</code> , branch in <code>run.ts</code> (<code>createRun</code> , <code>spawnRound</code> , <code>handleCut</code> , <code>restartRun</code>), a menu button in <code>ui/ui.ts</code> + <code>onPlay</code> in <code>index.ts</code> , best score key in <code>profile.ts</code> , tests
Change menu look/text	<code>ui/ui.ts</code> (structure) and <code>ui/styles.css</code> (look; colours also in <code>view/palette.ts</code>)
Change sounds	<code>sfx.ts</code> (frequencies, durations)
Change the ad rules	<code>rules.ts</code> <code>shouldShowInterstitial</code> , <code>config.interstitial</code>

11. Ship

```
npm run check && npm run build
npm run android:setup      # once (JDK 21 + Android SDK) - already done in this repo, android/ is
                             committed
npm run android            # run on a device
```

Real AdMob: see engine guide section 12; this game uses **rewarded** and **interstitial** units (`VITE_ADMOB_REWARDED_*`, `VITE_ADMOB_INTERSTITIAL_*`). Link for friends: engine guide section 10.

Still to verify on a real phone: touch feel and the knife offset under a finger, shop scrolling, real ad flows + consent/ATT, vibration and sound, performance on a low-end device.